

CLmodel Package: Overview and Examples

Anthony Almudevar¹

July 28, 2019

1. Department of Biostatistics and Computational Biology, University of Rochester

Contents

1	Model Definition	2
1.1	COP model definition	2
1.2	Distributional properties of the response vector	3
1.3	Exchangeable correlation	3
2	Model Fitting and Estimation - Theoretical Overview	3
2.1	Quasi-likelihood estimation	3
2.1.1	Maximum Likelihood for independent responses	4
2.1.2	Model estimation for correlated responses	4
2.2	Confidence bounds for fitted curves	5
3	Model Fitting and Estimation - Using the CLmodel Package	5
3.1	Utilities for formatting predictor variables	6
3.2	Using the main model function <code>fit.CL</code>	7
3.3	Initial values	9
3.4	Using the main fitting function <code>fit.CL</code>	10
3.5	Using the COP fit option	11
3.6	The <code>prediction</code> method for CL models	15
4	Compound Logistic Model with Two Varying Asymptotes	17
4.1	Simulation utilities	18
4.2	GOF test	20
4.3	Fitting the model	20
5	Modeling Correlation	22
5.1	Model Selection	27
5.2	Ridge Regression and Cross-Validation	28

1 Model Definition

The R package `CLmodel` implements the compound logistic binary model described in Almudevar and Pichichero (2019). We are given a vector of binary responses $\mathbf{y} = (y_1, \dots, y_n)$ (all vectors are interpretable as column vectors in the context of linear algebra).

`> library(CLmodel)`

The general form of an m component model is given by

$$\pi_i = P(y_i = 1) = H(\pi_{i1}, \dots, \pi_{im}), \quad i = 1, \dots, n, \quad (1)$$

where H is a function of m variables, which possesses all continuous first-order partial derivatives. We assume the model depends on a vector of parameters $\boldsymbol{\beta} \in \mathbb{R}^p$. Then let $\mathbf{x}_{ik} \in \mathbb{R}^{p_k}$, $k = 1, \dots, m$, be m predictor vectors paired with response y_i . The components of the model are then

$$\pi_{ik} = \phi(\boldsymbol{\beta}_k^T \mathbf{x}_{ik}), \quad k = 1, \dots, m, \quad (2)$$

where $\phi(u) = (1 + \exp(-u))^{-1}$ is the logistic function.

Note that this is the most general form of the model, in the sense that a single element β_j of $\boldsymbol{\beta}$ may appear in more than one component. However, it may be more natural to regard each component π_{ik} as defining a distinct logistic regression model with parameter vector $\boldsymbol{\beta}_k \in \mathbb{R}^{p_k}$ and predictor vector $\mathbf{u}_{ik} \in \mathbb{R}^{p_k}$, $k = 1, \dots, m$, where the number of predictors p_k associated with the components may vary. This type of model can be expressed in the form (2) by setting

$$\boldsymbol{\beta} = [\boldsymbol{\beta}_1 \mid \boldsymbol{\beta}_2 \mid \dots \mid \boldsymbol{\beta}_m]$$

and

$$\mathbf{x}_{ik} = [\mathbf{0} \mid \dots \mid \mathbf{u}_{ik} \mid \dots \mid \mathbf{0}], \quad k = 1, \dots, m,$$

where the subvector $\mathbf{u}_{ik}$ of $\mathbf{x}_{ik}$ occupies the same columns as does the subvector $\boldsymbol{\beta}_k$ of $\boldsymbol{\beta}$. We use the general form throughout, using the preceding conversion if needed.

Thus, each element y_i is associated with m predictor vectors $\mathbf{x}_{ik} \in \mathbb{R}^p$, $k = 1, \dots, m$. Then $\mathbf{X}_k$ is the $n \times p$ matrix with i th row equal to $\mathbf{x}_{ik}$.

For convenience, let $\theta \in \Theta \subset \mathbb{R}^q$ be the collection of all model parameters. We can write interchangeably $\pi_i = \pi_i(\theta)$, $\pi_{ik} = \pi_{ik}(\theta)$, $i = 1, \dots, n$; $k = 1, \dots, m$. We may also express the response function in vector form, in particular, $\tilde{\pi} = \tilde{\pi}(\theta) = (\pi_1(\theta), \dots, \pi_n(\theta))$, $\tilde{\pi}_k = \tilde{\pi}_k(\theta) = (\pi_{1k}(\theta), \dots, \pi_{nk}(\theta))$, $k = 1, \dots, m$.

1.1 COP model definition

The COP model is a slight modification of the component-wise linear model defined by (1)-(2). In addition to the response vector $\mathbf{y}$, parameter vector $\boldsymbol{\beta}$ and predictor matrices $\mathbf{X}_k$ defined above, a *COP variable* $\mathbf{z} = (z_1, \dots, z_n)$ and parameter $\alpha \in \mathbb{R}$ are introduced into the first component

$$\pi_{i1} = \phi(g(\alpha)(z_i - \boldsymbol{\beta}_1^T \mathbf{x}_{i1})), \quad (3)$$

where $g(\alpha)$ is a positive increasing function of α , and z_i represents the appropriate element of $\mathbf{z}$ (in the `CLmodel` package $g(\alpha) = \exp(\alpha)$ is used). The other components remain the same as (2). Here, the parameter θ includes α as well as the coefficient vector $\boldsymbol{\beta}$.

1.2 Distributional properties of the response vector

The choice of model fitting method is primarily determined by the correlation structure of the responses. We let V_θ be the $n \times n$ covariance matrix of the response vector $\mathbf{y}$, where $\theta \in \Theta \subset \mathbb{R}^q$ is the collection of all model parameters.

Independent responses. Let D_θ be $n \times n$ diagonal matrix with i th diagonal element $[D_\theta]_{ii} = \tau_i = \pi_i(1 - \pi_i)$, Then $V_\theta = D_\theta$.

Correlated responses. Let R be $n \times n$ correlation matrix of response vector y . Then

$$V_\theta = D_\theta^{1/2} R D_\theta^{1/2} \quad (4)$$

Note that considerable simplification follows when R does not depend on θ . If R is itself unknown (as is usually the case), a two-step procedure may be followed. An initial estimate of θ may be calculated, provisionally assuming independent responses, permitting an estimate of R . Then R may be used to refine the estimate of θ , and is in any case necessary for the estimation of standard errors. It is usually a practical necessity to adopt some parametric form for R .

1.3 Exchangeable correlation

Currently, `CLmodel` implements an exchangeable correlation model for response vector $\mathbf{y}$. Groups are assumed to be defined by levels of an n -dimensional factor `group`. Within-group responses are assumed to have pairwise correlation ρ .

2 Model Fitting and Estimation - Theoretical Overview

A number of model fitting methods are available in `CLmodel`.

2.1 Quasi-likelihood estimation

The model defined in Section 1 is not in general an exponential family model, and so methods associated with generalized linear models (GLM) or generalized estimating equations cannot be applied directly. However, quasi-likelihood methods (Wedderburn, 1974), on which much of the preceding theory is based, may be used, leading to a similar set of inference methods. Suppose we are given responses $\mathbf{y}$, with mean vector $\tilde{\pi}(\theta)$ and covariance matrix V_θ dependent on parameter $\theta \in \Theta \subset \mathbb{R}^q$. Let W be an $n \times n$ positive definite weight matrix. Let $\hat{\theta}_{QL}$ be the unique solution (if it exists) to the q -dimensional system of equations

$$G(\theta) = \Gamma_\theta^T W^{-1} [\mathbf{y} - \tilde{\pi}(\theta)] = \vec{0}. \quad (5)$$

where

$$\Gamma_\theta = \left[\frac{\partial \tilde{\pi}(\theta)}{\partial \theta_1} \cdots \frac{\partial \tilde{\pi}(\theta)}{\partial \theta_m} \right].$$

Note that W may depend on θ . It can be shown that as $n \rightarrow \infty$, $\hat{\theta}_{QL}$ exists with probability approaching 1, with asymptotic distribution

$$\hat{\theta} \sim N(\theta, \Sigma_\theta^W) \quad (6)$$

where

$$\Sigma_\theta^W = [\Gamma_\theta^T W^{-1} \Gamma_\theta]^{-1} [\Gamma_\theta^T W^{-1} V_\theta W^{-1} \Gamma_\theta] [\Gamma_\theta^T W^{-1} \Gamma_\theta]^{-1}. \quad (7)$$

Note that (7) yields the important special case

$$\Sigma_\theta = \Sigma_\theta^{V_\theta} = [\Gamma_\theta^T V_\theta^{-1} \Gamma_\theta]^{-1}. \quad (8)$$

Inference proceeds using the approximation $\Sigma_\theta^W \approx \Sigma_{\hat{\theta}_{QL}}^W$.

2.1.1 Maximum Likelihood for independent responses

For independent responses the maximum likelihood function is

$$L(\theta) = \sum_{i=1}^n y_i \log(\pi_i) + (1 - y_i) \log(1 - \pi_i), \quad (9)$$

It is easily shown that where the maximum likelihood estimate

$$\hat{\theta}_{MLE} = \arg \max_\theta L(\theta) \quad (10)$$

is a stationary point of $L(\theta)$, it satisfies the system of equations (5) where $W = D_\theta$. Therefore, the asymptotic distribution of θ_{MLE} is given by (6) after substituting $W = D_\theta$.

2.1.2 Model estimation for correlated responses

A number of procedures are available when responses y are correlated, that is, the covariance matrix V_θ is given by (4) where R is a general covariance matrix. The quasi-likelihood method is sufficiently flexible in that the maximum likelihood estimate $\hat{\theta}_{MLE}$, based on likelihood function (9), may be used even if independence does not hold. This is because it is equivalent to the quasi-likelihood estimate, formally the solution $\hat{\theta}_{QL}$ to equations (5) where $W = D_\theta$, which then possesses asymptotic distribution

$$\hat{\theta}_{QL} \sim N(\theta, \Sigma_\theta^{D_\theta}).$$

Estimation of $\Sigma_\theta^{D_\theta}$ requires an estimate of V_θ , which in turn requires an estimate $\hat{R} \approx R$. Given estimate $\hat{\theta}_{QL}$, we may calculate estimates of the mean and variance $\hat{\pi}_i$, $\hat{\tau}_i$ for each response y_i , yielding Pearson residuals

$$\hat{r}_i = \frac{y_i - \hat{\pi}_i}{\hat{\tau}_i^{1/2}}. \quad (11)$$

The residuals can be used to calculate estimate $\hat{R} \approx R$. A parametric model is almost always used for this (Section 1.3). At this point, $\hat{R}$ is taken as fixed, giving

$$V_\theta \approx D_\theta^{1/2} \hat{R} D_\theta^{1/2}. \quad (12)$$

This approximation, with D_θ , is substituted into (7), so that the covariance of $\hat{\theta}_{QL}$ is estimated by

$$\Sigma_\theta^{D_\theta} \approx \Sigma_{\hat{\theta}_{QL}}^{D_\theta}.$$

It is also possible, once $\hat{R}$ is obtained in this way, to set W in (5) equal to the approximation (12), in order to recalculate the quasi-likelihood estimate, yielding $\hat{\theta}'_{QL}$. In this case, the covariance matrix of the estimate is calculated using (8) and (12), yielding

$$\Sigma_\theta \approx \Sigma_{\hat{\theta}'_{QL}}.$$

2.2 Confidence bounds for fitted curves

There will be interest in estimating curves $\pi(z; x)$, interpreted as the expected response $E[y]$ as a function of the COP variable $z \in \mathbb{R}$, with any remaining covariates $x \in \mathbb{R}^q$, $q \geq 1$, held fixed. The asymptotic marginal variance is given by

$$\sigma_{\hat{\pi}(z;x)}^2 = \left[\frac{\partial \pi(z; x)}{\partial \theta} \right]^T \Sigma_{\theta} \left[\frac{\partial \pi(z; x)}{\partial \theta} \right]. \quad (13)$$

The package `CLmodel` implements marginal confidence bands of user specified confidence level. In Cox and Ma (1995) it is shown that a simultaneous $1 - \alpha$ confidence band results when the critical value $(\chi_{m,\alpha}^2)^{1/2}$ from a χ^2 distribution with m degrees of freedom is used.

3 Model Fitting and Estimation - Using the CLmodel Package

To demonstrate the various functions of the `CLmodel` package, we will introduce a relatively straightforward model. Let $\mathbf{y} = (y_1, \dots, y_n)$ be a binary response vector, and let $\tilde{x} = (x_1, \dots, x_n)$ be a single predictor variable. We first consider the standard logistic regression model,

$$\pi_i = \phi(\beta_0 + \beta_1 x_i), \quad i = 1, \dots, n. \quad (14)$$

Of course, this model has the usual asymptotes at 0 and 1. Suppose instead we model the asymptotes 0 and $\lambda \in (0, 1)$, where λ is a parameter to be estimated. This is easily done by incorporating the parameter λ into (14), yielding the modified model

$$\pi_i = \lambda \times \phi(\beta_1 + \beta_2 x_i). \quad (15)$$

Of course, this is a simple example of a two-component compound logistic model. Using the notation of Section 1 we have

$$\begin{aligned} \boldsymbol{\beta} &= (\beta_1, \beta_2, \beta_3), \\ \mathbf{x}_{i1} &= (1, x_i, 0), \\ \mathbf{x}_{i2} &= (0, 0, 1), \end{aligned} \quad (16)$$

which define the components

$$\begin{aligned} \pi_{i1} &= \phi(\boldsymbol{\beta}^T \mathbf{x}_{i1}) = \phi(\beta_1 + \beta_2 x_i), \\ \pi_{i2} &= \lambda = \phi(\boldsymbol{\beta}^T \mathbf{x}_{i2}) = \phi(\beta_3), \end{aligned} \quad (17)$$

so that λ is estimated using the log-odds parametrization $\beta_3 = \log(\lambda/(1 - \lambda))$.

When fitting a compound logistic model, the function H which defines the model (1) must be given explicitly, as well as its partial derivatives, in this case,

$$\begin{aligned} H(\pi_1, \pi_2) &= \pi_1 \times \pi_2, \\ \frac{\partial H}{\partial \pi_1} &= \pi_2, \\ \frac{\partial H}{\partial \pi_2} &= \pi_1. \end{aligned} \quad (18)$$

3.1 Utilities for formatting predictor variables

`CLmodel` contains a few simple utilities to convert predictor variables into the correct format. Suppose in the current example we have $n = 4$ samples, with predictor variable $\tilde{x} = (1.2, 3.4, 7.6, 8.2)$. Then the design matrices for the components would be

$$\mathbf{X}_1 = \begin{bmatrix} 1 & 1.2 & 0 \\ 1 & 3.4 & 0 \\ 1 & 7.6 & 0 \\ 1 & 8.2 & 0 \end{bmatrix} \quad \text{and} \quad \mathbf{X}_2 = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

The function `fit.CL.make.xmatrices` accepts a list of m matrices, where m is the number of model components. The k th matrix is an $n \times p_k$ matrix with column vectors representing predictor variables associated with the k th component. This should include the intercept term if needed. Note that this function cannot be used when a parameter appears in more than one component. However, the same predictor variable may be used in more than one component.

```
> ###
> ### Predictor variable
> ###
>
> x = c(1.2, 3.4, 7.6, 8.2)
> ###
> ### Intercept
> ###
>
> x0 = rep(1,4)
> ###
> ### Create design matrices in format required by CLmodel functions
> ###
>
> x.list = list(cbind(x0,x),x0)
> fit.CL.make.xmatrices(x.list)

[[1]]
      [,1] [,2] [,3]
[1,]     1  1.2    0
[2,]     1  3.4    0
[3,]     1  7.6    0
[4,]     1  8.2    0

[[2]]
      [,1] [,2] [,3]
[1,]     0    0    1
[2,]     0    0    1
[3,]     0    0    1
[4,]     0    0    1
```

We may also create specialized parameter labels with the function `fit.CL.covariate.labels`, with subscripts indicating component index and parameter index within component.

```
> ### There are 2 parameters for component 1, and 1 parameter for component 2.
>
> pattern = c(2,1)
> fit.CL.covariate.labels(pattern, cop.flag = FALSE)

[1] "b[1,1]" "b[1,2]" "b[2,1]"
```

This gives alternative indexing conventions $(\beta_1, \beta_2, \beta_3) = (\beta[1, 1], \beta[1, 2], \beta[2, 1])$.

3.2 Using the main model function `fit.CL`

To demonstrate the main model function `fit.CL` we load the data list `sampleDataCLmodel` included in the `CLmodel` package.

```
> data(sampleDataCLmodel)
> names(sampleDataCLmodel)

[1] "y"           "x.cop"       "x.covariates" "h.func"      "h.prime.func"
[6] "cop.flag"

> length(sampleDataCLmodel$y)

[1] 1000
```

This object contains data, with $n = 1000$ observations, which may be used to fit the two-component model defined by Equations (16)-(17). The response vector is contained in `sampleDataCLmodel$y`. The single predictor variable is stored in `sampleDataCLmodel$x.cop`. However, this variable is already incorporated in the list of design matrices `sampleDataCLmodel$x.covariates`, in which the rows are formatted according to (16):

```
> head(sampleDataCLmodel$x.covariates[[1]])

      [,1]      [,2] [,3]
[1,]    1 0.8336722    0
[2,]    1 0.5754634    0
[3,]    1 0.2864519    0
[4,]    1 -0.5931442    0
[5,]    1 0.5412552    0
[6,]    1 0.2855030    0

> head(sampleDataCLmodel$x.covariates[[2]])

      [,1] [,2] [,3]
[1,]    0    0    1
[2,]    0    0    1
```

```
[3,] 0 0 1
[4,] 0 0 1
[5,] 0 0 1
[6,] 0 0 1
```

We may create labels with the `fit.CL.covariate.labels` utilities:

```
> est.labs = fit.CL.covariate.labels(c(2,1))
> est.labs
```

```
[1] "b[1,1]" "b[1,2]" "b[2,1]"
```

The model function $H(\pi_1, \pi_2)$ and its derivatives (see Equation (18)) are included in the list `sampleDataCLmodel` as R functions:

```
> sampleDataCLmodel$h.func

function(p) {p[1]*p[2]}
<bytecode: 0x7fd0e63ec978>

> sampleDataCLmodel$h.prime.func

$hdev1
function (p)
{
  p[2]
}
<bytecode: 0x7fd0e63ec208>

$hdev2
function (p)
{
  p[1]
}
<bytecode: 0x7fd0e63ebbe8>
```

The function H is represented by the list element `sampleDataCLmodel$h.func`, which is a function that accepts a numeric vector of length 2, then evaluates and outputs the value of H . The partial derivatives are stored as a list of functions `sampleDataCLmodel$h.prime.func` (one function for each component), each of which accepts a numeric vector of length 2, then evaluates and outputs the value of the corresponding partial derivative.

Finally, we note that where needed we set the value `sampleDataCLmodel$cop.flag = FALSE`, since we are not using the COP model defined in Section 1.1.

3.3 Initial values

An initial estimate for the parameter β must be supplied. The `CLmodel` package contains a utility function `fit.CL.init` for generating an initial heuristic fit from which an initial parameter value may be constructed. The initial model is based on a response vector $\mathbf{y}$ and a single predictor variable $\tilde{x} = (x_1, \dots, x_n)$ of the same length, and takes the form

$$\pi_i = a_1[1 - \phi(\alpha(x_i - x_{thr}))] + a_2\phi(\alpha(x_i - x_{thr})), \quad i = 1, \dots, n$$

where $\theta = (\alpha, a_1, a_2, x_{thr})$ is the four dimensional parameter to be estimated. The response is essentially a logistic response function in which the asymptotes 0 and 1 have been replaced by a_1 and a_2 . The convention used here is to constrain $\alpha > 0$, so that a_1 is the asymptote approached as $x_i \rightarrow -\infty$. This loses no generality, since the response function is increasing (or decreasing) if $a_2 > a_1$ (or $a_2 < a_1$).

The function `fit.CL.init` requires the response vector and a single predictor variable. In most cases, the parameter $\theta = (\alpha, a_1, a_2, x_{thr})$ fit by the function `fit.CL.init` will have to be transformed to conform to the appropriate model parametrization defined in Section 1. For example, the model (16)-(17) uses a three-dimensional parameter $\beta = (\beta_1, \beta_2, \beta_3)$. The reparametrization would take the form

$$\begin{aligned} \beta_1 &= -\alpha x_{thr} \\ \beta_2 &= \alpha \\ \beta_3 &= \log(a_2/(1 - a_2)) \end{aligned}$$

if $a_2 > a_1$, and

$$\begin{aligned} \beta_1 &= \alpha x_{thr} \\ \beta_2 &= -\alpha \\ \beta_3 &= \log(a_1/(1 - a_1)) \end{aligned}$$

if $a_1 > a_2$. Since it is assumed that one asymptote is 0, only $\max\{a_1, a_2\}$ is used for the initial estimate. The function `fit.CL.init` optionally accepts a function `init.function` which accepts four numerical parameters $(\alpha, a_1, a_2, x_{thr})$ and outputs a single reparametrized vector β . If `init.function` is not provided, a vector of initial estimates of parameters $(\alpha, a_1, a_2, x_{thr})$ will be output.

In the current example, using the sample data list `sampleDataCLmodel`, an initial estimate would be produced as follows:

```
> init.obj = fit.CL.init(sampleDataCLmodel$y, sampleDataCLmodel$x.cop)
> init.obj

      alpha      a1      a2      x.thr
4.9999321 0.1853040 0.6709966 -0.1518398

> init.est = c(-init.obj[4]*init.obj[1],
+             init.obj[1],
+             log(init.obj[3]/(1-init.obj[3])))
> init.est
```

x.thr	alpha	a2
0.7591888	4.9999321	0.7126958

Here, it was anticipated that $a_2 > a_1$.

3.4 Using the main fitting function `fit.CL`

We now have all the elements needed to fit a compound logistic model. Continuing with the current example the fit is produced in the following way:

```
> fit.CL.obj <- fit.CL(y = sampleDataCLmodel$y,
+                       x.cop = NULL,
+                       x.covariates = sampleDataCLmodel$x.covariates,
+                       group = NULL,
+                       initial.estimate = init.est,
+                       h.func = sampleDataCLmodel$h.func,
+                       h.prime.func = sampleDataCLmodel$h.prime.func,
+                       cop.flag = FALSE,
+                       exchangeable = FALSE,
+                       quasi.lik = FALSE,
+                       lambda = 0.0,
+                       linear.comb = NULL,
+                       est.labs = est.labs,
+                       control = list(reltol = 1e-14))
> ###
> ### Generic summary function for S3 class = fit.CL
> ###
>
> summary(fit.CL.obj)
```

	Est	SE	Z	p-val (Z)
b[1,1]	1.05029	0.28945	3.6285	2.8504e-04
b[1,2]	3.74336	0.49559	7.5533	4.2435e-14
b[2,1]	0.99619	0.15770	6.3170	2.6677e-10

Maximum-likelihood estimate:

Independence assumed

N = 1000 ; model DF = 3

Log-lik = -566.4751 ; AIC = 1138.95 ; BIC = 1153.673

```
> ###
> ### Simple fit function
> ###
>
> fit.CL.simple(y = sampleDataCLmodel$y,
+               x.cop = NULL,
+               x.covariates = sampleDataCLmodel$x.covariates,
```

```

+           group = NULL,
+           initial.estimate = init.est,
+           h.func = sampleDataCLmodel$h.func,
+           h.prime.func = sampleDataCLmodel$h.prime.func,
+           cop.flag = FALSE,
+           exchangeable = FALSE,
+           quasi.lik = FALSE,
+           lambda = 0.0,
+           est.labs = est.labs,
+           control = list(reltol = 1e-14))

$est
  b[1,1]  b[1,2]  b[2,1]
1.050289 3.743360 0.996186

$cor.est
NULL

>

```

Since the option `cop.flag = FALSE` is selected, the COP model is not fit (Section 1.1), so the argument `x.cop` need not be provided. Also note that a simple version of `fit.CL` is included, which outputs only the parameter estimates.

The responses are assumed to be independent, so the option `exchangeable = FALSE` is specified. Currently, only the exchangeable correlation model is supported, which would also require input of the group variable `group`. If this is not provided, the responses are assumed to be independent, regardless of the value of the `exchangeable` option.

The `quasi.lik` option is used when the exchangeable correlation model is requested. The option `lambda` is used to define ridge regression model fits. If this parameter is zero, then the unpenalized least squares fit is calculated.

The `fit.CL` function outputs an S3 object of class `fit.CL`, for which a generic summary function `summary.fit.CL` is included in the `CLmodel` package, which provides the standard parameter estimate table.

3.5 Using the COP fit option

It is possible to reparametrize the model (16)-(17) as a COP model (Section (1.1)):

$$\pi = \lambda \times \phi(e^{\alpha}(x - x_{thr})).$$

To fix ideas, we are assuming that π is an increasing function of what is now the COP variable x . If $(\beta'_1, \beta'_2, \beta'_3)$ are the parameters of Equation (17), the parametric transformation between the COP

and non-COP models is then

$$\begin{aligned}\alpha &= \log(\beta'_2), \\ x_{thr} &= -\beta'_1/\beta'_2, \\ \lambda &= \phi(\beta'_3).\end{aligned}$$

Under the COP model parametrization the variable x is not included in the component predictor matrices, which are now the $n \times 2$ matrices

$$\mathbf{X}_1 = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ \vdots & \vdots \\ 1 & 0 \end{bmatrix} \quad \text{and} \quad \mathbf{X}_2 = \begin{bmatrix} 0 & 1 \\ 0 & 1 \\ \vdots & \vdots \\ 0 & 1 \end{bmatrix}.$$

Other than α , the COP model components are then models the threshold parameter, that is,

$$\begin{aligned}x_{thr} &= \boldsymbol{\beta}^T \mathbf{x}_1 \\ \log(\lambda/(1-\lambda)) &= \boldsymbol{\beta}^T \mathbf{x}_2,\end{aligned}$$

where $\mathbf{x}_i$ is a row of $\mathbf{X}_i$, $i = 1, 2$.

Specialized labels for the COP model can be obtained using the `cop.flag = TRUE` option:

```
> est.labs = fit.CL.covariate.labels(c(1,1), cop.flag = TRUE)
> est.labs

[1] "alpha" "b[1,1]" "b[2,1]"
```

Note that the scale parameter α is included as the first element of parameter $\theta = (\alpha, \beta_1, \beta_2)$.

An initial estimate can now be obtained as follows

```
> init.obj = fit.CL.init(sampleDataCLmodel$y, sampleDataCLmodel$x.cop)
> init.obj

      alpha      a1      a2      x.thr
4.9999321 0.1853040 0.6709966 -0.1518398

> init.est = c(log(init.obj[1]),
+             init.obj[4],
+             log(init.obj[3]/(1-init.obj[3])))
> init.est

      alpha      x.thr      a2
1.6094243 -0.1518398 0.7126958
```

The prediction matrices now only include intercept terms in the appropriate columns, but these must be explicitly constructed.

```

> ###
> ### Sample size
> ###
>
> data(sampleData)
> n = length(sampleDataCLmodel$y)
> ###
> ### Intercept term
> ###
>
> x0 = rep(1,n)
> ###
> ### Create design matrices in format required by CLmodel functions
> ###
>
> x.list = list(x0, x0)
> x.covariates = fit.CL.make.xmatrices(x.list)
> head(x.covariates[[1]])

```

```

      [,1] [,2]
[1,]     1     0
[2,]     1     0
[3,]     1     0
[4,]     1     0
[5,]     1     0
[6,]     1     0

```

```

> head(x.covariates[[2]])

```

```

      [,1] [,2]
[1,]     0     1
[2,]     0     1
[3,]     0     1
[4,]     0     1
[5,]     0     1
[6,]     0     1

```

Refit the model with option `cop.flag = TRUE`:

```

> fit.CL.obj.COP <- fit.CL(y = sampleDataCLmodel$y,
+                           x.cop = sampleDataCLmodel$x.cop,
+                           x.covariates = x.covariates,
+                           group = NULL,
+                           initial.estimate = init.est,
+                           h.func = sampleDataCLmodel$h.func,
+                           h.prime.func = sampleDataCLmodel$h.prime.func,
+                           cop.flag = TRUE,

```

```

+             exchangeable = FALSE,
+             quasi.lik = FALSE,
+             lambda = 0.0,
+             linear.comb = NULL,
+             est.labs = est.labs,
+             control = list(reltol = 1e-14))
> ###
> ### Fit summary
> ###
>
> summary(fit.CL.obj.COP)

```

	Est	SE	Z	p-val (Z)
alpha	1.31998	0.132392	9.9702	2.0574e-23
b[1,1]	-0.28057	0.050409	-5.5660	2.6065e-08
b[2,1]	0.99619	0.157700	6.3170	2.6677e-10

Maximum-likelihood estimate:

Independence assumed

N = 1000 ; model DF = 3

Log-lik = -566.4751 ; AIC = 1138.95 ; BIC = 1153.673

```

> ###
> ### summary for non-COP model
> ###
>
> summary(fit.CL.obj)

```

	Est	SE	Z	p-val (Z)
b[1,1]	1.05029	0.28945	3.6285	2.8504e-04
b[1,2]	3.74336	0.49559	7.5533	4.2435e-14
b[2,1]	0.99619	0.15770	6.3170	2.6677e-10

Maximum-likelihood estimate:

Independence assumed

N = 1000 ; model DF = 3

Log-lik = -566.4751 ; AIC = 1138.95 ; BIC = 1153.673

The summary from the previous (non-COP) model is included above for comparison. As expected, the parameter `b[2,1]` has the same interpretation for both models, and the estimate is the same value 0.99619 for each fit. For the COP model, `b[1,1]` = -0.28057 is the estimate of x_{thr} . This parameter can be expressed using the non-COP model parameters by $x_{thr} = -\beta_1/\beta_2$, estimated by `-b[1,1]/b[1,2]` = -1.0503/3.7434 = -0.28057, using the `cop.flag = FALSE` option. This is the same estimate obtained using the `cop.flag = TRUE` option. For the COP model the estimate of α is `alpha` = 1.32. This parameter is expressed using the non-COP model by $\alpha = \log(\beta_2)$. The estimate of β_2 from the non-COP fit is `b[1,2]` = 3.7434, which provides the equivalent estimate $\log(1.32) = 0.27762$ of α .

3.6 The prediction method for CL models

The `CLmodel` package contains a `predict` method for the `fit.CL` class output by the `fit.CL` function. This method calculates fitted values for the fitted model summarized in a `fit.CL` class object. If argument `newdata = NULL`, the fitted values will be those corresponding to the original predictor variables. Alternatively, fitted values for new predictor variables can be calculated by including them in the argument `newdata`. If the `cop.flag = TRUE` option was used to produce the `fit.CL` class object then `newdata` must be a list of the form `list(x.cop, x.covariates)`. Otherwise, `newdata` can be either a list with an element labelled `x.covariates`, or a `covariates` type object itself.

In either case, the `predict` method outputs an $n \times 4$ matrix, where n is the number of design points. The first two columns contain the fitted values and their standard errors, while the last two columns contain lower and upper $1 - \alpha$ confidence bounds for the fit. The option `alpha.crit` is used to specify α , with default `alpha.crit = 0.05`. The critical value used to construct the confidence bounds is $(\chi^2_{p,\alpha})^{1/2}$, where p is the model degrees of freedom (Cox and Ma, 1995). In the next example, the fitted curve is plotted using an evenly spaced grid.

```
> library(CLmodel)
> data(sampleDataCLmodel)
> ###
> ### fit using regular model
> ###
>
> est.labs = fit.CL.covariate.labels(c(2,1))
> init.obj = fit.CL.init(sampleDataCLmodel$y, sampleDataCLmodel$x.cop)
> init.est = c(-init.obj[4]*init.obj[1],
+             init.obj[1],
+             log(init.obj[3]/(1-init.obj[3])))
> fit.CL.obj <- fit.CL(y = sampleDataCLmodel$y,
+                     x.cop = NULL,
+                     x.covariates = sampleDataCLmodel$x.covariates,
+                     group = NULL,
+                     initial.estimate = init.est,
+                     h.func = sampleDataCLmodel$h.func,
+                     h.prime.func = sampleDataCLmodel$h.prime.func,
+                     cop.flag = FALSE,
+                     exchangeable = FALSE,
+                     quasi.lik = FALSE,
+                     lambda = 0.0,
+                     linear.comb = NULL,
+                     est.labs = est.labs,
+                     control = list(reltol = 1e-14))
> ###
> ### Fit summary
> ###
```

```

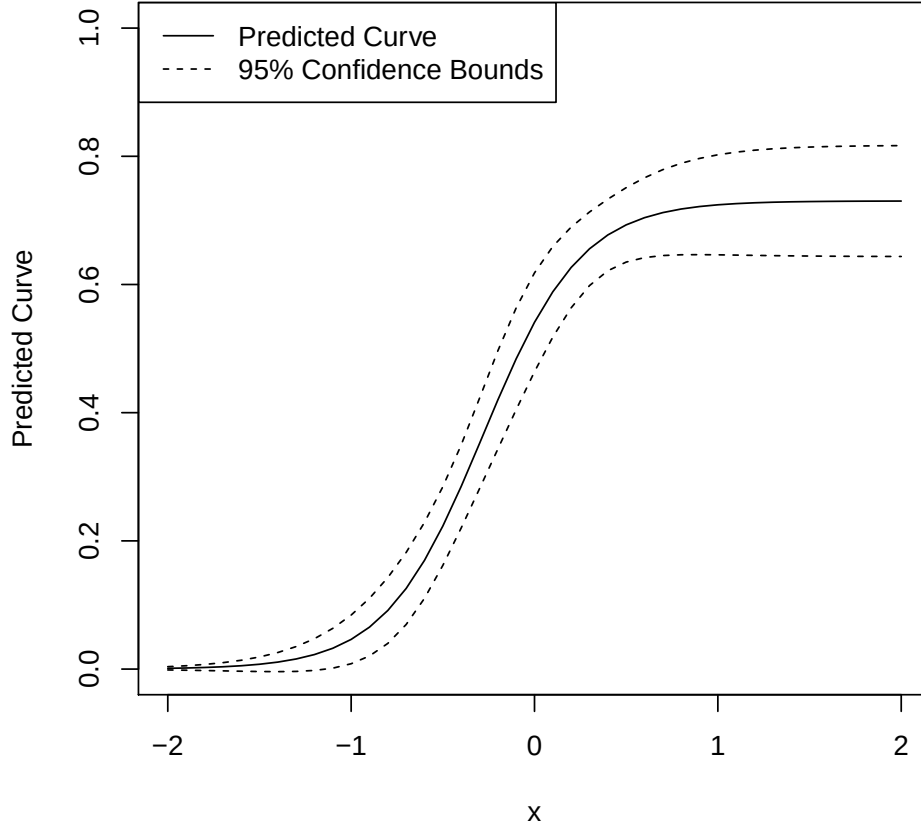
>
> summary(fit.CL.obj)

              Est      SE      Z  p-val (Z)
b[1,1] 1.05029 0.28945 3.6285 2.8504e-04
b[1,2] 3.74336 0.49559 7.5533 4.2435e-14
b[2,1] 0.99619 0.15770 6.3170 2.6677e-10

Maximum-likelihood estimate:
Independence assumed
N = 1000 ; model DF = 3
Log-lik = -566.4751 ; AIC = 1138.95 ; BIC = 1153.673

> ###
> ### Construct grid on which to plot fitted curve
> ###
>
> xgrid = seq(-2, 2, 0.1)
> ###
> ### Intercept
> ###
>
> x0 = rep(1,length(xgrid))
> ###
> ### Create design matrices
> ###
>
> x.list = list(cbind(x0,xgrid),x0)
> x.covariates = fit.CL.make.xmatrices(x.list)
> ###
> ### calculate then plot fitted curve on grid
> ###
>
> mu.est = predict(fit.CL.obj, newdata = x.covariates)
> matplot(xgrid, mu.est[,c(1,3,4)], ylim = c(0,1), lty = c(1,2,2),
+         col = 'black',type = 'l', xlab = 'x',ylab = 'Predicted Curve')
> legend('topleft',legend=c('Predicted Curve','95% Confidence Bounds'),
+       lty=1:2,col='black')

```



4 Compound Logistic Model with Two Varying Asymptotes

In this section we consider a second example which will demonstrate additional features of the COP package. Let x be a single predictor variable. Then consider the response function

$$\pi = a_1 [1 - \phi(b + cx)] + a_2 \phi(b + cx)$$

where $a_1, a_2 \in [0, 1]$. It is important to first note that the model is not uniquely specified. If b, c are multiplied by -1 then and a_1, a_2 exchanged, the resulting model is identical. We may equivalently use the COP model,

$$\pi = a_1 [1 - \phi(e^\alpha(x - x_{thr}))] + a_2 \phi(e^\alpha(x - x_{thr})), \quad (19)$$

which is identical, but removes this ambiguity. Now, a_1, a_2 are the lower and upper asymptotes, and π is an increasing or decreasing function of x when $a_2 > a_1$ or $a_1 < a_2$.

To express (19) as a compound logistic model, we set

$$\pi = H(\pi_1, \pi_2, \pi_3) = \pi_2(1 - \pi_1) + \pi_3\pi_1, \quad (20)$$

where

$$\begin{aligned}\pi_1 &= \phi(e^\alpha(x - \beta_1)) \\ \pi_2 &= a_1 = \phi(\beta_2), \\ \pi_3 &= a_2 = \phi(\beta_3).\end{aligned}$$

The model parameters are now $\theta = (\alpha, \beta_1, \beta_2, \beta_3)$.

A natural inference problem would be to detect and estimate differences between the asymptotes a_1, a_2 . A natural method for this is to estimate the linear combination

$$\beta_3 - \beta_2 = \log(a_2/(1 - a_2)) - \log(a_1/(1 - a_1))$$

which is simply the log-odds ratio for a_2 and a_1 . The `fit.CL` function will provide estimates and standard errors for any linear combinations of parameters.

4.1 Simulation utilities

A simulation utility is included with the `CLmodel` package. Here, we will create a simulation for the model (19). We need to create an object representing the compound logistic function H given in (20) and its derivatives, as follows:

```
> h.func = function(p) {p[2]*(1-p[1]) + p[3]*p[1]}
> hdev1 = function(p) {-p[2]+p[3]}
> hdev2 = function(p) {1-p[1]}
> hdev3 = function(p) {p[1]}
> h.prime.func = list(hdev1 = hdev1, hdev2 = hdev2, hdev3 = hdev3)
```

We can then create a COP variable with $n = 1000$, confined to the interval $x \in [-3, 3]$:

```
> set.seed(543876)
> n.sample = 1000
> x.cop = runif(n.sample, -3, 3)
```

The predictor matrices are constructed as follows:

```
> x1 = rep(1, n.sample)
> x.covariates = fit.CL.make.xmatrices(list(x1, x1, x1))
```

Since the `cop.flag = TRUE` option will be used, the COP variable will be a separate object. In our simulation, we will assume $\alpha = 3$, $x_{thr} = 1$, $a_1 = 0.75$, $a_2 = 0.1$. The parameter vector will be of the form

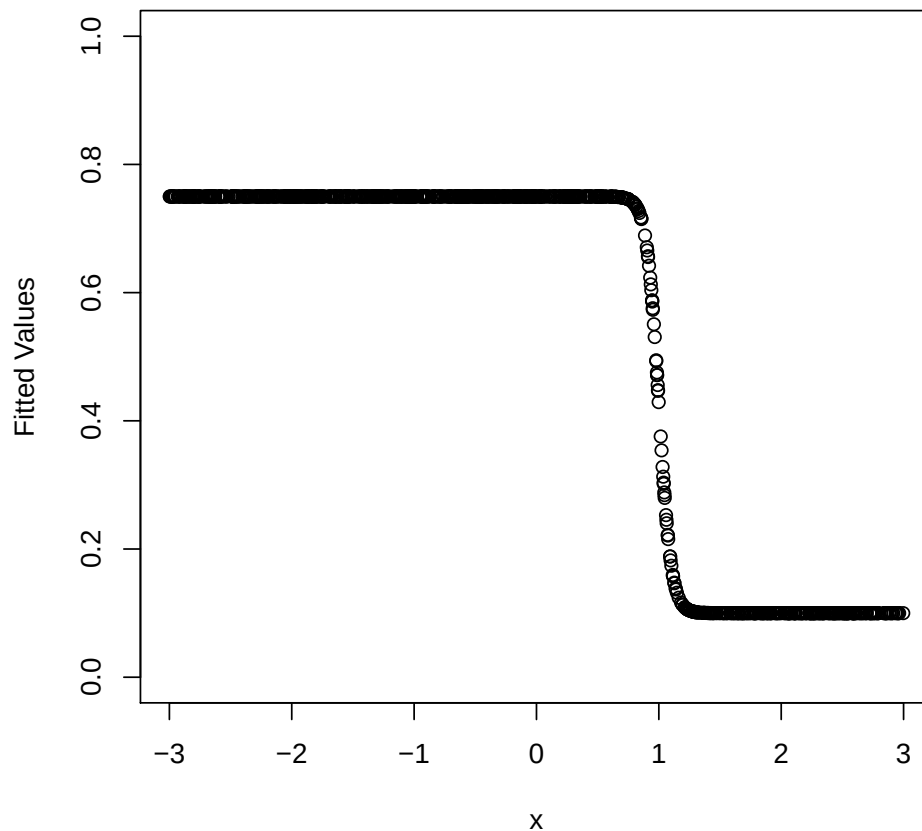
$$\theta = (\alpha, x_{thr}, \log(a_1/(1 - a_1)), \log(a_2/(1 - a_2))) = (\alpha, \beta_1, \beta_2, \beta_3).$$

In order to simulate data for this model, the fitted values must first be calculated, which may be done using the `fit.CL.fitted.values` function. This function accepts as input a parameter vector and a list containing the remaining model elements.

```

> true.par = c(3,1,log(0.75/0.25),log(0.1/0.9))
> parms = list(x.cop = x.cop,
+             x.covariates = x.covariates,
+             h.func = h.func,
+             h.prime.func = h.prime.func,
+             cop.flag = TRUE)
> mu = fit.CL.fitted.values(true.par,parms)$mu
> plot(x.cop, mu, ylim = c(0,1), xlab = 'x', ylab = 'Fitted Values')

```



The fitted values may then be used to simulate a response vector using the `fit.CL.sim.data` function:

```

> ymat = fit.CL.sim.data(mu,1)
> y = ymat[1,]

```

Here, no correlation model is used, so the responses are independent. Furthermore, only a single replication was requested. Whatever the number of replications requested, the `fit.CL.sim.data` function outputs a matrix, each row representing a single replication.

4.2 GOF test

A goodness-of-fit test is implemented in the function `fit.CL.GOF` included with the `CLmodel`. The null model is standard logistic regression. A response vector `y` and predictor variables `x.data` must be included. The `x.data` object may be a numeric vector, matrix or data frame. An exchangeable correlation model may be used if a group variable `group` is included. The null distribution is generated by bootstrap replications from an estimated standard logistic model with response `y` and all predictor variables included in `x.data`. The number of replications is given by the argument `nboot`. Note that the initial estimate is included in the bootstrap replications, so that the minimum estimated P -value is, appropriately, $P = 1/nboot$. The GOF function is demonstrated below:

```
> fit.CL.GOF.obj = fit.CL.GOF(y,x.cop)
> fit.CL.GOF.obj
```

```
$chi.stat.obs
[1] 1029.39
```

```
$chisq.df
[1] 0 998
```

```
$p.value
[1] 0.001998002
```

```
$rho.est
$rho.est$est
NULL
```

```
$rho.est$se
NULL
```

The estimated P -value is $P = 0.001998002$, so we may reject the standard logistic regression model.

4.3 Fitting the model

We can use the main `fit.CL` function to fit the model, while including an estimate for $\beta_3 - \beta_2$. Estimates for linear combinations of the parameters can be requested using the `linear.comb` argument of `fit.CL`. This is a $L \times p$ matrix, where L is the number of linear combinations, and p is the number of parameters. Each row of `linear.comb` represents a linear combination. The model parameter is $\theta = (\alpha, \beta_1, \beta_2, \beta_3)$, so for our example, the `linear.comb` argument would be the single row matrix

$$A = [0 \ 0 \ -1 \ 1]$$

```
> ###
> ### Creat labels
> ###
>
> est.labs = fit.CL.covariate.labels(c(1,1,1),cop.flag = TRUE)
> est.labs
```

```
[1] "alpha" "b[1,1]" "b[2,1]" "b[3,1]"
```

```
> ###
> ### Create matrix of linear combinations (labels can be introduced as row labels for this mat
> ###
>
> linear.comb = matrix(c(0,0,-1,1),nrow = 1)
> rownames(linear.comb) = c('b[3,1] - b[2,1]')
> ###
> ### Get initial estimate
> ###
>
> init.obj = fit.CL.init(y,x.cop)
> lodds = function(x) {log(x/(1-x))}
> init.est = c(log(init.obj[1]),
+             init.obj[4],
+             lodds(init.obj[2]),
+             lodds(init.obj[3]))
> ###
> ### create \ttt{fit.CL} class object containing fit
> ###
>
> fit.CL.obj <- fit.CL(y = y,
+                      x.cop = x.cop,
+                      x.covariates = x.covariates,
+                      group = NULL,
+                      initial.estimate = init.est,
+                      h.func = h.func,
+                      h.prime.func = h.prime.func,
+                      cop.flag = TRUE,
+                      exchangeable = FALSE,
+                      quasi.lik = FALSE,
+                      lambda = 0.0,
+                      linear.comb = linear.comb,
+                      est.labs = est.labs,
+                      control = list(reltol = 1e-14))
> ###
> ### Fit summary
> ###
>
> summary(fit.CL.obj)
```

	Est	SE	Z	p-val (Z)
alpha	2.69639	0.460407	5.8565	4.7261e-09
b[1,1]	0.97149	0.035689	27.2211	3.6518e-163
b[2,1]	1.01345	0.090126	11.2448	2.4553e-29

```

b[3,1]          -2.16284 0.196147 -11.0267  2.8420e-28
b[3,1] - b[2,1] -3.17629 0.217197 -14.6240  1.9741e-48

```

Maximum-likelihood estimate:

Independence assumed

N = 1000 ; model DF = 4

Log-lik = -509.3383 ; AIC = 1026.677 ; BIC = 1046.308

5 Modeling Correlation

Currently, the `CLmodel` package supports an exchangeable correlation model. We will also include an additional covariate. Assume $n = 1000$, consisting of 5 repeated measures from 200 subjects. An exchangeable correlation parameter of $\rho = 0.1$ will be assumed. In addition, assume the age of each subject is given as z (uniformly distributed on the interval (20,75)).

The following code simulates a COP variable, and the subject ages. It is advisable to center any covariate at 0, so that the coefficient estimate is independent of the intercept estimate.

```

> library(CLmodel)
> set.seed(543876)
> n.sample = 1000
> x.cop = runif(n.sample, -3, 3)
> agei = runif(200, 20, 75)
> agea = rep(agei, each = 5)
> agec = agea - mean(agea)
> sub.id = rep(1:200, each = 5)
>

```

The model can be written, using double subscripts to identify the model components,

$$\pi = H(\pi_1, \pi_2, \pi_3) = \pi_2(1 - \pi_1) + \pi_3\pi_1, \quad (21)$$

where

$$\begin{aligned} \pi_1 &= \phi(e^\alpha(x - \beta_{11} - \beta_{12}z)) \\ \pi_2 &= \phi(\beta_{21} + \beta_{22}z), \\ \pi_3 &= \phi(\beta_{31} + \beta_{32}z). \end{aligned}$$

The model parameter is now $\theta = (\alpha, \beta_{11}, \beta_{12}, \beta_{21}, \beta_{22}, \beta_{31}, \beta_{32})$. The covariate z is included in each model component, although the associated coefficient is different in each (as discussed above, it would be possible to constrain these coefficients to be equal, or otherwise not linearly independent). It is interesting to note that the threshold parameter x_{thr} is now an embedded linear model $x_{thr} =$

$\beta_{11} + \beta_{12}z$. Here we will take at the true parameters

$$\begin{aligned}\alpha &= 1, \\ (\beta_{11}, \beta_{12}) &= (1, 1/25), \\ (\beta_{21}, \beta_{22}) &= (\log(0.75/0.25), 0), \\ (\beta_{31}, \beta_{32}) &= (\log(0.25/0.75), -1/25).\end{aligned}$$

Next, set up the model data:

```
> ###
> ### Model function
> ###
>
> h.func = function(p) {p[2]*(1-p[1]) + p[3]*p[1]}
> hdev1 = function(p) {-p[2]+p[3]}
> hdev2 = function(p) {1-p[1]}
> hdev3 = function(p) {p[1]}
> h.prime.func = list(hdev1 = hdev1, hdev2 = hdev2, hdev3 = hdev3)
> ###
> ### Format predictor variables
> ###
>
> x1 = rep(1,n.sample)
> x2 = agec
> x.obj = fit.CL.make.xmatrices(list(cbind(x1,x2),cbind(x1,x2),cbind(x1,x2)))
> est.labs = fit.CL.covariate.labels(c(2,2,2), cop.flag = TRUE)
> parms = list(x.cop = x.cop,
+             x.covariates = x.obj,
+             h.func = h.func,
+             h.prime.func = h.prime.func,
+             group = sub.id,
+             cop.flag = TRUE)
```

Next, simulate model data

```
> true.par = c(1,0,1/25,log(0.75/0.25),0,log(0.25/0.75),-1/25)
> rho = 0.1
> mu = fit.CL.fitted.values(true.par,parms)$mu
> ymat = fit.CL.sim.data(mu,1,sub.id,rho)
> y = ymat[1,]
```

Do a GOF test.

```
> fit.CL.GOF(y, cbind(x.cop, agec), group = sub.id, nboot = 50)

$chi.stat.obs
[1] 1006.17
```

```

$chisq.df
[1] -1 995

$p.value
[1] 0.09803922

$rho.est
$rho.est$est
[1] 0.1029613

$rho.est$se
[1] 0.02933718

```

To construct an initial solution, assume the coefficients associated with covariate z equal zero (if this is done, covariates should be centered at zero).

```

> init.obj = fit.CL.init(y,x.cop)
> init.est = c(log(init.obj[1]),
+             init.obj[4],
+             0,
+             lodds(init.obj[2]),
+             lodds(init.obj[3]), 0)
> ###
> ### Model assuming independent responses
> ###
>
> fit.CL.obj.u = fit.CL(y = y,
+                       x.cop = x.cop,
+                       x.covariates = x.obj,
+                       group = sub.id,
+                       initial.estimate = true.par,
+                       h.func = h.func,
+                       h.prime.func = h.prime.func,
+                       cop.flag = T,
+                       exchangeable = F,
+                       quasi.lik = F,
+                       lambda = 0.0,
+                       linear.comb = NULL,
+                       est.labs = est.labs,
+                       control = list(reltol = 1e-14))
> summary(fit.CL.obj.u)

```

	Est	SE	Z	p-val (Z)
alpha	1.103218	0.3878520	2.8444	4.4491e-03
b[1,1]	0.156062	0.1533123	1.0179	3.0871e-01

```

b[1,2]  0.054802 0.0092522  5.9232 3.1576e-09
b[2,1]  0.894800 0.1383924  6.4657 1.0085e-10
b[2,2] -0.014899 0.0080006 -1.8623 6.2566e-02
b[3,1] -1.021946 0.1807921 -5.6526 1.5804e-08
b[3,2] -0.040953 0.0105653 -3.8762 1.0612e-04

```

Maximum-likelihood estimate:

Independence assumed

N = 1000 ; model DF = 7

Log-lik = -628.3014 ; AIC = 1270.603 ; BIC = 1304.957

```

> ###
> ### Ordinary least squares fit. Estimates are constructed assuming
> ### independence with standard errors incorporating the
> ### correlation model.
> ###
>
> fit.CL.obj.ex = fit.CL(y = y,
+                         x.cop = x.cop,
+                         x.covariates = x.obj,
+                         group = sub.id,
+                         initial.estimate = true.par,
+                         h.func = h.func,
+                         h.prime.func = h.prime.func,
+                         cop.flag = T,
+                         exchangeable = T,
+                         quasi.lik = F,
+                         lambda = 0.0,
+                         linear.comb = NULL,
+                         est.labs = est.labs,
+                         control = list(reltol = 1e-14))
> summary(fit.CL.obj.ex)

```

	Est	SE	Z	p-val (Z)
alpha	1.103218	0.3879118	2.8440	4.4552e-03
b[1,1]	0.156062	0.1529619	1.0203	3.0760e-01
b[1,2]	0.054802	0.0092749	5.9087	3.4478e-09
b[2,1]	0.894800	0.1452105	6.1621	7.1793e-10
b[2,2]	-0.014899	0.0085329	-1.7461	8.0794e-02
b[3,1]	-1.021946	0.1898679	-5.3824	7.3498e-08
b[3,2]	-0.040953	0.0114408	-3.5795	3.4422e-04

Maximum-likelihood estimate:

Exchangeable Correlation: rho = 0.1044

Number of groups = 200

N = 1000 ; model DF = 7

```
Log-lik = -628.3014 ; AIC = 1270.603 ; BIC = 1304.957
```

```
> ###
> ### Quasilikelihood method.
> ###
>
> fit.CL.obj.ql = fit.CL(y = y,
+                         x.cop = x.cop,
+                         x.covariates = x.obj,
+                         group = sub.id,
+                         initial.estimate = true.par,
+                         h.func = h.func,
+                         h.prime.func = h.prime.func,
+                         cop.flag = T,
+                         exchangeable = T,
+                         quasi.lik = T,
+                         lambda = 0.0,
+                         linear.comb = NULL,
+                         est.labs = est.labs,
+                         control = list(reltol = 1e-14))
> summary(fit.CL.obj.ql)
```

	Est	SE	Z	p-val (Z)
alpha	1.171974	0.3776585	3.1033	1.9140e-03
b[1,1]	0.168808	0.1436711	1.1750	2.4001e-01
b[1,2]	0.053330	0.0086383	6.1737	6.6717e-10
b[2,1]	0.885664	0.1384860	6.3953	1.6020e-10
b[2,2]	-0.012554	0.0082258	-1.5262	1.2695e-01
b[3,1]	-1.052325	0.1843465	-5.7084	1.1404e-08
b[3,2]	-0.043425	0.0111441	-3.8966	9.7538e-05

Quasi-likelihood estimate:

Exchangeable Correlation: rho = 0.1044

Number of groups = 200

N = 1000 ; model DF = 7

Log-lik = -628.3014 ; QIC = 1271.838 ; QICu = 1270.848

The three fitting methods differ by the settings of the logical arguments `exchangeable` and `quasi.lik`. Because the responses are correlated, the model `fit.CL.obj.u` fit with options `exchangeable = FALSE` and `quasi.lik = FALSE` (the independent response model) is clearly inappropriate. Model `fit.CL.obj.ex` is fit with options `exchangeable = TRUE` and `quasi.lik = FALSE`. This model is fit assuming independent responses, however, the standard errors incorporate estimated correlation. Accordingly, the parameter estimates for models `fit.CL.obj.u` and `fit.CL.obj.ex` are identical. However, the standard errors differ. As would be expected, those in model `fit.CL.obj.ex` are generally larger, since they account for subject level correlation.

The estimating equations used for model `fit.CL.obj.ql` differ from the other two fits, so the estimates differ, although not substantially. This method is more efficient for correlated data than ordinary least squares method, so the standard errors tend to be smaller, and so is the recommended method for correlated responses.

Note that a common method of estimating correlation ρ is used when `exchangeable = TRUE`. In our example, this estimate is 0.10442, close to the true value $\rho = 0.1$.

5.1 Model Selection

The AIC and BIC scores are calculated for the independent response model, and QIC and QICu scores are calculated for the quasiliikelihood model (AIC and BIC are not interpretable for correlated data). Here, we continue the current example, constructing a fit which deletes the age

```
> ###
> ### Include only intercept terms in x.covariates object
> ###
>
> x.obj = fit.CL.make.xmatrices(list(x1,x1,x1))
> est.labs = fit.CL.covariate.labels(c(1,1,1), cop.flag = TRUE)
> ###
> ### reformat initial solution
> ###
>
> lodds = function(x) {log(x/(1-x))}
> true.par.000 = c(log(init.obj[1]),
+                  init.obj[4],
+                  lodds(init.obj[2]),
+                  lodds(init.obj[3]))
> ###
> ### Fit model without age covariate
> ###
>
> fit.CL.obj.ql.000 = fit.CL(y = y,
+                            x.cop = x.cop,
+                            x.covariates = x.obj,
+                            group = sub.id,
+                            initial.estimate = true.par.000,
+                            h.func = h.func,
+                            h.prime.func = h.prime.func,
+                            cop.flag = T,
+                            exchangeable = T,
+                            quasi.lik = T,
+                            lambda = 0.00,
+                            linear.comb = NULL,
+                            est.labs = est.labs,
```

```
+                               control = list(reltol = 1e-14))
> summary(fit.CL.obj.q1.000)
```

	Est	SE	Z	p-val (Z)
alpha	1.54584	0.52120	2.9659	3.0179e-03
b[1,1]	-0.17267	0.12937	-1.3347	1.8197e-01
b[2,1]	0.87361	0.12708	6.8748	6.2093e-12
b[3,1]	-0.61691	0.11274	-5.4718	4.4550e-08

Quasi-likelihood estimate:

Exchangeable Correlation: rho = 0.09277

Number of groups = 200

N = 1000 ; model DF = 4

Log-lik = -638.9309 ; QIC = 1288.532 ; QICu = 1286.765

In the full model QIC = 1271.838, QICu = 1270.848, and in the reduced model QIC = 1288.532, QICu = 1286.765, so by the QIC score, the full model is preferred.

5.2 Ridge Regression and Cross-Validation

The CLmodel package supports ridge regression. The least-squares criterion is modified to

$$\Gamma = SSE + \lambda \theta^T \theta.$$

An equivalent form is used for the quasi-likelihood method. The penalty parameter λ is specified by the `lambda` argument in the `fit.CL` or `fit.CL.simple` function. This parameter may be selected using cross-validation using the `fit.CL.cv` function. Apart from those arguments used for `fit.CL` function, the K -fold parameter must be supplied in the `k.cv` argument. If `k.cv = NULL` then $K = n$, the sample size of the data, which is equivalent to leave-one-out cross-validation (LOOCV). Three GOF scores are available. The argument `lambda` can be a vector of values for which cross-validated scores will be calculate.

```
> x.obj = fit.CL.make.xmatrices(list(cbind(x1,x2),cbind(x1,x2),cbind(x1,x2)))
> est.labs = fit.CL.covariate.labels(c(2,2,2), cop.flag = TRUE)
> lambda.list = seq(0,1,0.1)
> fit.CL.cv.obj = fit.CL.cv(y = y,
+                               x.cop = x.cop,
+                               x.covariates = x.obj,
+                               group = sub.id,
+                               initial.estimate = true.par,
+                               h.func = h.func,
+                               h.prime.func = h.prime.func,
+                               cop.flag = T,
+                               exchangeable = T,
+                               quasi.lik = T,
+                               lambda = lambda.list,
+                               k.cv = 2,
```

```

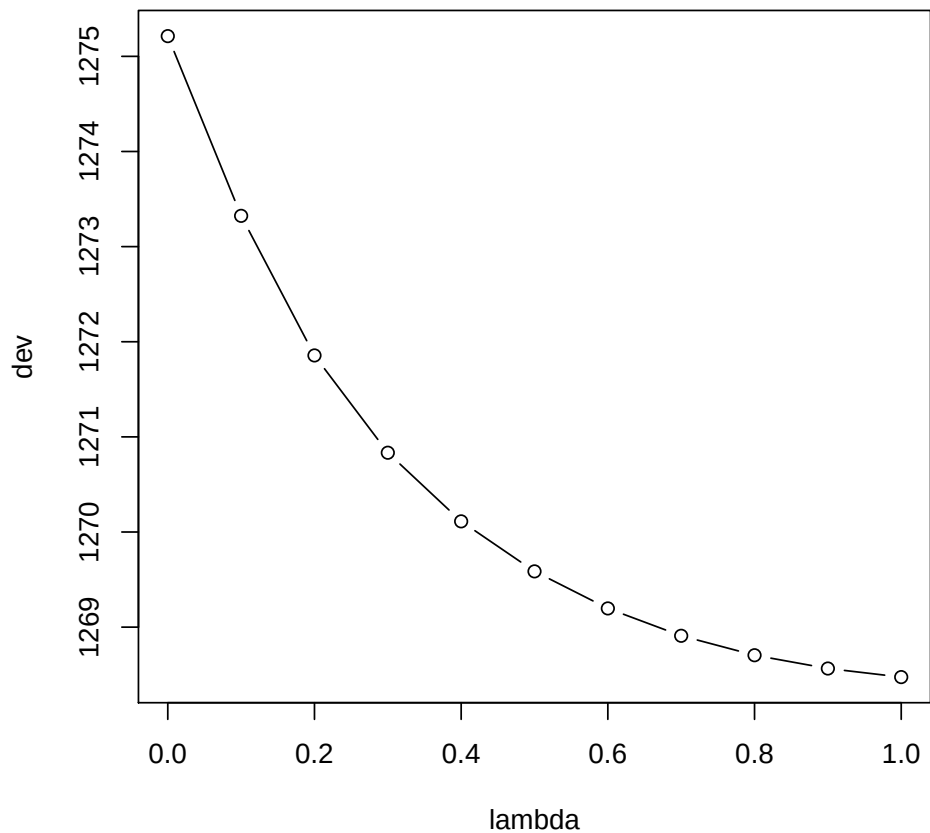
+                                     gof.score.type = 'dev',
+                                     est.labs = est.labs,
+                                     control = list(reltol = 1e-14))
> ###
> ### display table, and plot dev against lambda
> ###
>
> fit.CL.cv.obj

$cv.best.dev
      lambda      dev
      1.000 1268.475

$cv.table
      lambda      dev
1      0.0 1275.213
2      0.1 1273.324
3      0.2 1271.856
4      0.3 1270.834
5      0.4 1270.112
6      0.5 1269.586
7      0.6 1269.197
8      0.7 1268.909
9      0.8 1268.705
10     0.9 1268.565
11     1.0 1268.475

> plot(fit.CL.cv.obj$cv.table,type = 'b',xlab = 'lambda',ylab = 'dev')

```



References

- A. Almudevar and M. Pichichero. Binary response models for correlate of protection studies. *JRSS-C (under revision)*, 2019.
- C. Cox and G. Ma. Asymptotic confidence bands for generalized nonlinear regression models. *Biometrics*, 51:142–150, 1995.
- R. W. M. Wedderburn. Quasi-likelihood functions, generalized linear models, and the Gauss—Newton method. *Biometrika*, 61(3):439–447, 1974.